

LAB TASK 3

Design, Simulate and Synthesize a Digital Access Control System

Hamming Distance Detector

Digital Access control system is nothing but a hamming distance detector. By definition, the Hamming distance between two binary codes of equal length is the number of positions at which the corresponding bits are different. If the hamming distance is zero, this means that the input value matches the corresponding secret code. Similarly if the hamming distance is greater than 0, this means that the input value is incorrect.

This architecture forms the foundational logic for multi-tier security authentication, error-correcting memory (ECC), and digital communication systems. It is also used in telecommunication networks for error detection and error correction.

Hardware Design:

Hamming distance detector is simply a bitwise XOR operation because XOR gate gives logic “1” when the inputs are different. So, an N-bit secret code will match an N-bit entered code, iff all the bits are same resulting 0 after the XOR operation.

Design Specifications:

Inputs : Secret Code, Entered Code

Outputs: Unlock, Warning, Alarm

If hamming distance = 0, Unlock = 1.

If hamming distance = 1, Warning = 1.

If hamming distance > 1, Alarm = 1.

RTL:

```
module hd_detector(
input logic [3:0]entered_code,
input logic [3:0]secret_code,
output logic unlock,
output logic warning,
output logic alarm
);

logic [3:0]hd;
logic [2:0]diff;

assign hd = secret_code ^ entered_code;
```

```
assign diff = hd[3] + hd[2] + hd[1] + hd[0]; // count no of ones in the
hamming distance
```

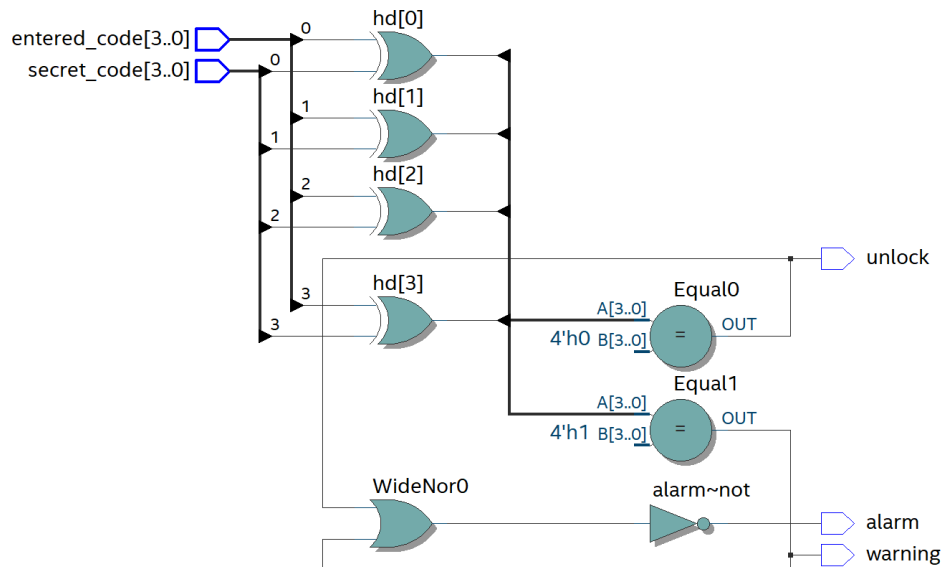
```
always_comb begin
    unlock      = 0;
    warning     = 0;
    alarm       = 0;
    case(hd)
        3'b0:      unlock      = 1;
        3'b1:      warning     = 1;
        default: alarm  = 1; // when difference is greater than 1
    endcase
end
endmodule
```

RTL Explanation:

Rather than writing out a massive, error-prone 16-case lookup table, the circuit leverages a highly scalable structural arithmetic approach:

- **Mismatch Extraction:** A bitwise XOR operation ($\text{secret_code} \wedge \text{entered_code}$) is performed. Because an XOR gate outputs 1 only when inputs differ, it generates a "fault vector."
- **Count number of ones(Bit Addition):** The individual bits of the XOR result are added together ($\text{hd}[3] + \text{hd}[2] + \text{hd}[1] + \text{hd}[0]$). This arithmetic sum yields the exact number of mismatched bits (ranging from 0 to 4).
- **Output Classification:** A compact combinational case block maps the final bit count into three mutually exclusive security alerts:
 - **0 Mismatches:** Triggers unlock (Perfect match).
 - **1 Mismatch:** Triggers warning (Filters minor human typos).
 - **2 or more Mismatches:** Triggers alarm (Flags intrusive guess attempts).

RTL Diagram:



Logic Utilization Summary:

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri Jun 26 21:09:51 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	hd_detector
Top-level Entity Name	hd_detector
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	3 / 56,480 (< 1 %)
Total registers	0
Total pins	11 / 268 (4 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	0 / 156 (0 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Testbench:

```

`timescale 1ns/1ps
module hd_detector_tb;

logic [3:0]entered_code;
logic [3:0]secret_code;
logic unlock;
logic warning;
logic alarm;

//dut instantiation
hd_detector dut(
.entered_code(entered_code),
.secret_code(secret_code),
.unlock(unlock),
.warning(warning),
.alarm(alarm)
);

// Test cases
initial begin
    secret_code = 4'b1010; entered_code = 4'b1010; #10; // Unlock
    secret_code = 4'b1010; entered_code = 4'b1011; #10; // Warning
    secret_code = 4'b1010; entered_code = 4'b1001; #10; // Alarm
    secret_code = 4'b0101; entered_code = 4'b0101; #10; // Unlock
    secret_code = 4'b0101; entered_code = 4'b1101; #10; // Warning
    secret_code = 4'b0101; entered_code = 4'b1010; #10; // Unlock
$stop;
end
endmodule

```

Waveforms: